جزوه درجه یک سیستم عامل

فصل ۳ - بخش ۸: ارتباط در سیستم‌های کارفرما-کارگزار

در بخش‌های قبلی، با مدل‌های IPC (حافظه مشترک، عبور پیغام، لوله‌ها) آشنا شدیم که بیشتر برای ارتباط پردازه‌ها *روی یک کامپیوتر* بودند.

حالا می‌خواهیم ببینیم ارتباط در مقیاس بزرگتر (بین دو کامپیوتر مختلف در یک شبکه) یا در سیستم‌های مدرن کارفرما-کارگزار (Client-Server) چگونه کار می‌کند. دو راهبرد اصلی اینجا وجود دارد: **سوکت‌ها** و **فراخوانی روال دور (RPC)**.

۳-۸-۱: سوکت‌ها (Sockets)

تشبیه: سوکت مثل یک "پریز برق" یا "خط تلفن" برای شبکه است.

تعریف دقیق: یک **"نقطه انتهایی" (Endpoint) برای ارتباط** است.

اگر دو پردازه در شبکه بخواهند با هم صحبت کنند، هر کدام باید یک "سوکت" باز کنند. ارتباط از طریق این جفت سوکت برقرار می‌شود.

آدرس‌دهی (چطور سوکت را پیدا کنیم؟): یک سوکت با دو چیز شناسایی می‌شود:

1. **آدرس IP:** (مثل آدرس خیابان ساختمان) - مشخص می‌کند کدام *کامپیوتر* در شبکه.
2. **شماره پورت (Port):** (مثل شماره واحد یا آپارتمان) - مشخص می‌کند کدام *پردازه* روی آن کامپیوتر.

مدل کارفرما-کارگزار (Client-Server):

- **کارگزار (Server):**
  1. روی یک "پورت" مشخص شروع به **"گوش دادن" (Listen)** می‌کند.
  2. فراخوانی `accept()` را صدا می‌زند و **مسدود (Block)** می‌شود (منتظر می‌ماند).
- **کارفرما (Client):**
  1. با دانستن IP و پورت سرور، یک درخواست **"اتصال" (Connect)** می‌فرستد.
- **اتصال:**
  1. سرور از حالت مسدود خارج می‌شود و تابع `accept()` یک *سوکت جدید* (مخصوص آن کارفرما) برمی‌گرداند.
  2. حالا سرور می‌تواند با کارفرما از طریق این سوکت جدید صحبت کند و همزمان به "گوش دادن" روی پورت اصلی برای کارفرماهای بعدی ادامه دهد.

انواع شماره پورت‌ها (مهم):

- **پورت‌های شناخته‌شده (Well-known Ports):** پورت‌های **زیر ۱۰۲۴** که برای سرویس‌های استاندارد رزرو شده‌اند. (مثلاً SSH: **22**، FTP: **21**، HTTP: **80**).

- **پورت‌های موقت (Ephemeral Ports):** پورت‌های **بالای ۱۰۲۴** که سیستم عامل به صورت *تصادفی* به کارفرماها (Clients) اختصاص می‌دهد تا بتوانند به سرور وصل شوند.

مثال: یک کلاینت روی کامپیوتر `146.86.5.20` می‌خواهد به وب‌سرور `161.25.19.8` وصل شود.

- **سوکت سرور:** `161.25.19.8:80` (IP سرور + پورت ۸۰)
- **سوکت کلاینت:** `146.86.5.20:1625` (IP کلاینت + یک پورت تصادفی مثل ۱۶۲۵)

این "جفت سوکت" اتصال را **یکتا** می‌کند.

آدرس ویژه `127.0.0.1` (Loopback): این آدرس IP همیشه به معنی "خودِ همین کامپیوتر" است. برای تست کردن سرور و کلاینت روی یک دستگاه استفاده می‌شود.

نقد سوکت‌ها: سوکت‌ها "سطح پایین" هستند. آن‌ها فقط یک "جریان" (Stream) از بایت‌های خام را منتقل می‌کنند. اینکه این بایت‌ها چه معنایی دارند (آیا عدد هستند، یا متن، یا عکس) بر عهده برنامه‌نویس است.

۳-۸-۲: فراخوانی‌های روال دور (RPC - Remote Procedure Calls)

تشبیه: RPC مثل "کنترل از راه دور جادویی" است.

ایده اصلی: کاری کنیم که فراخوانی یک تابع روی یک کامپیوتر *دیگر* (در شبکه)، دقیقاً مثل فراخوانی یک تابع *محلی* به نظر برسد. این یک **"تجرید" (Abstraction)** سطح بالا است.

جادوی RPC: "تِه‌کد" ها (Stubs) این تمام چیزی است که RPC را ممکن می‌سازد.

- **تِه‌کد کارفرما (Client Stub):**
  - این یک تابع "قلابی" روی دستگاه کلاینت است که اسمش شبیه تابع اصلی سرور است.
  - وقتی کلاینت آن را صدا می‌زند (مثلاً `result = server.add(5, 3)`):
    1. تِه‌کد کلاینت پارامترها (۵ و ۳) را **"بسته‌بندی" (Marshal)** می‌کند و در یک پیغام می‌گذارد.
    2. این پیغام را با استفاده از سوکت به سرور می‌فرستد.
    3. منتظر جواب می‌ماند.
- **تِه‌کد کارگزار (Server Stub):**
  1. پیغام را از کلاینت دریافت می‌کند.
  2. آن را **"بازگشایی" (Unmarshal)** می‌کند و پارامترها (۵ و ۳) را بیرون می‌کشد.
  3. تابع *واقعی* `add(5, 3)` را روی سرور صدا می‌زند.
  4. نتیجه (مثلاً ۸) را دوباره "بسته‌بندی" (Marshal) کرده و برای کلاینت پس می‌فرستد.

کلاینت جواب (۸) را دریافت می‌کند و اصلاً نمی‌فهمد که این تابع در آن سوی دنیا اجرا شده است!

چالش‌های RPC (و راه‌حل‌ها):

۱. چالش بسته‌بندی (Marshalling):

- کامپیوترها داده‌ها را متفاوت ذخیره می‌کنند. (مثلاً "Big-Endian" در مقابل "Little-Endian").

- **راه‌حل:** استفاده از یک فرمت استاندارد و مستقل از ماشین مثل **XDR (External Data Representation)**. هر دو طرف قبل از ارسال، داده‌ها را به XDR تبدیل می‌کنند و بعد از دریافت، از XDR به فرمت خودشان برمی‌گردانند.

۲. چالش معنایی (Semantics):

- در شبکه ممکن است پیغام گم شود یا تکراری ارسال شود.
- **معنای "حداکثر یکبار" (At-most-once):** (رایج‌ترین)
  - به هر پیغام یک "مُهر زمانی" (Timestamp) یا ID یکتا می‌دهیم.
  - سرور لیستی از IDهایی که قبلاً پردازش کرده را نگه می‌دارد و پیغام‌های تکراری را نادیده می‌گیرد.
- **معنای "دقیقاً یکبار" (Exactly-once):** (ایده‌آل اما سخت)
  - این حالت نیازمند "حداکثر یکبار" + ارسال **"رسید" (ACK)** از سمت سرور است.
  - کلاینت باید آنقدر پیغام را دوباره بفرستد تا ACK آن را دریافت کند.

۳. چالش اتصال (Binding) - (پیدا کردن سرور):

- کلاینت از کجا بداند سرور روی کدام "پورت" گوش می‌دهد؟
- **راه‌حل ۱: پورت‌های ثابت (Static):** پورت‌ها از قبل تعیین شده و هاردکد می‌شوند (مثل پورت ۸۰ برای وب). ساده ولی انعطاف‌ناپذیر.
- **راه‌حل ۲: پورت‌های پویا (Dynamic) + دلال (Matchmaker):** (بهترین روش)
  1. یک سرویس "دلال" (Matchmaker) روی یک پورت *ثابت* (مثلاً ۱۱۸) اجرا می‌شود.
  2. سرور وقتی روشن می‌شود، یک پورت *تصادفی* (مثلاً ۴۵۶۷) می‌گیرد و به "دلال" می‌گوید: "سلام، من سرویس پرینت هستم روی پورت ۴۵۶۷".
  3. کلاینت به "دلال" (روی پورت ۱۱۸) پیغام می‌دهد: "سلام، شماره پورت سرویس پرینت چنده؟"
  4. "دلال" جواب می‌دهد: "پورت ۴۵۶۷".
  5. کلاینت حالا مستقیماً به سرور روی پورت ۴۵۶۷ وصل می‌شود.

۳-۸-۳: RPC در اندروید (یک مثال داخلی)

RPCها فقط برای شبکه نیستند. اندروید از RPC برای ارتباط *بین پردازه‌هایی که روی همان دستگاه* اجرا می‌شوند استفاده می‌کند.

- **سرویس (Service):** مؤلفه‌ای در اندروید که در پس‌زمینه اجرا می‌شود (مثل پخش موسیقی).
- **AIDL (زبان تعریف رابط اندروید):**
  - برنامه‌نویس فقط توابعی که می‌خواهد "از راه دور" قابل دسترسی باشند را در یک فایل `.aidl` تعریف می‌کند (مثلاً `remoteMethod()`).
  - ابزارهای اندروید **به صورت خودکار** تمام کدهای "تِه‌کد" (Stubs) کلاینت و سرور را تولید می‌کنند!
- **چارچوب Binder:** خودِ اندروید مدیریت "بسته‌بندی" (Marshalling) و ارسال پیغام بین دو پردازه را با استفاده از RPC انجام می‌دهد.

در نتیجه، برنامه‌نویس اندروید یک تابع را در پردازه دیگر صدا می‌زند، انگار که یک تابع محلی ساده است، و چارچوب Binder تمام پیچیدگی‌های RPC را در پشت صحنه انجام می‌دهد.